ONLINE FOOD ORDERING & DELIVERY SYSTEM

CS23531 – WEB PROGRAMMING
MINI PROJECT REPORT

Submitted by

Arun Prasath K – 241001018

Of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)



OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "**ONLINE FOOD ORDERING & DELIVERY SYSTEM"** is the Bonafide work of **ARUN PRASATH K (241001018)** who carried out the work under my supervision.

SIGNATURE
Dr.P. Neelaveni
HEAD OF THE DEPARTMENT
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam,
Chennai - 602105

SIGNATURE
Ms. Janani M
COURSE INCHARGE
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam,
Chennai - 602105

This project is submitted for CS23531 – Web Programming held on _____

INTERNAL EXAMINAR **EXTERNAL EXAMINAR**

ABSTRACT:

The Online Food Ordering and Delivery System is a full-stack web application developed using React, PHP, and MySQL. It provides a platform where customers can browse food items, view menus, add items to their cart, place orders, and check their order status. The React front end provides a responsive and user-friendly interface, PHP handles the server-side operations, and MySQL stores customer details, food items, order information, and payment details. The system allows users to register, log in, manage their carts, place orders, and view their order history. It also enables administrators to manage food items and update order statuses. By replacing traditional phone-based ordering with an online platform, the system simplifies the ordering process, improves order management, and provides customers with a convenient and efficient food delivery experience.

TABLE OF CONTENTS

S.No	Title	Page No
1.1	Introduction	4
1.2	Purpose	4
1.3	Scope of the project	4
1.4	Software requirement specification	5
2.1	Use case diagram	8
2.2	Entity-relationship diagram	8
2.3	Data flow diagram	9
3	Module description	10
4	Implementation	10
5	Conclusion	19
6	References	19

ONLINE FOOD ORDERING & DELIVERY SYSTEM

1.1 INTRODUCTION

The Online Food Ordering and Delivery System is a web-based application that allows customers to order food online from a variety of food items and menus. Traditionally, customers place orders through phone calls or by visiting restaurants, which can be time-consuming and inconvenient.

This project provides a digital platform where customers can browse food items, view prices and descriptions, add items to their cart, and place orders online. The system uses React for the user interface, PHP for server-side processing, and MySQL for storing customer details, food items, and order information. It simplifies the food ordering process and provides a convenient way for customers to order food and for administrators to manage food items and orders.

1.2 PURPOSE

The purpose of the system is to provide a simple and organised platform for online food ordering and delivery. It aims to:

- Replace traditional food ordering methods with an online platform.
- Allow customers to browse food items and view prices.
- Enable customers to add food items to their cart and place orders.
- Store customer, food, and order details in a MySQL database.
- Help administrators manage food items and customer orders.
- Improve the convenience and efficiency of the food ordering process.
- Live statistics: counts of total, lost, found and claimed notices, and a database connection indicator

1.3 SCOPE OF THE PROJECT

The Online Food Ordering and Delivery System is designed for customers and administrators to manage food ordering activities through a web application. Its scope covers:

- Food Menu: Displaying available food items with names, descriptions, prices, and images.
- Customer Management: Allowing customers to register, log in, and manage their details, if implemented.
- Cart Management: Adding food items to the cart, updating quantities, and removing items.

- Order Placement: Allowing customers to confirm their food selections and place orders online.
- Order Management: Enabling administrators to view customer orders and update their status.
- Database Management: Storing food items, customer information, and order details in a MySQL database.
- Responsive Interface: Providing a user-friendly interface that can be accessed through supported web browsers.

1.4 SOFTWARE REQUIREMENTS OF THE PROJECT

Overall Description

The Online Food Ordering and Delivery System allows customers to browse food items, add items to their cart, place orders, and view order details through a web browser. The React interface communicates with the PHP backend, which stores and retrieves information from the MySQL database. The system simplifies food ordering, improves order management, and provides a convenient platform for customers and administrators.

Product Perspective

The system follows a three-tier client-server architecture:

1. Presentation Tier: The React application with HTML5, CSS3, and JavaScript provides the user interface.
2. Application Tier: PHP handles server-side processing, customer requests, food item management, and order operations.
3. Data Tier: The MySQL database stores customer details, food items, cart information, and order records.

Hardware Requirements:

- Processor: Intel i3 or higher
- RAM: 4GB or higher
- Hard Disk: 500GB or higher
- OS: Windows 8, 10, 11

Software Requirements:

- Database: MySQL 8.0
- Frontend: React 18, HTML5, CSS3, JavaScript
- Backend: PHP 8.3

Product Functionality

a) **Food Menu:** Displays all available food items with their names, descriptions, prices, and images.

b) **Search Food:** Finds food items by searching for their names or keywords.

c) **Filter Food:** Filters food items based on categories such as starters, main courses, snacks, and beverages.

d) **View Food Details:** Displays complete information about selected food items, including descriptions and prices.

e) Add to Cart: Allows customers to add food items to their cart and select quantities.

f) **Manage Cart:** Allows customers to update item quantities, remove items, and calculate the total bill.

g) **Place Order:** Enables customers to confirm their food selections and submit orders.

h) **Order Management:** Allows administrators to view orders and update their status, such as Pending, Preparing, and Delivered.

i) **Customer Management:** Stores and manages customer details and order information using the MySQL database.

j) **Order History:** Allows customers to view their previous orders and check order details.

Assumptions and Dependencies

- The React environment, PHP runtime, and MySQL server must be installed and configured properly on the local machine.
- The application must run on a configured local or web server and be accessible through a web browser.
- An internet connection may be required to load external libraries and resources used by the React application.
- Customers are assumed to provide accurate information while registering, entering delivery addresses, and placing orders.
- The PHP backend must be configured correctly to communicate with the MySQL database.

Functional Requirements

1. Food Menu Management Module (FMM)

- The system displays available food items with their names, descriptions, prices, and images.
- Customers can browse food items and view their details.
- The administrator can add, update, and delete food items.

2. Search and Filter Module (SFM)

- Customers can search for food items using keywords or names.
- Food items can be filtered by categories such as starters, main courses, snacks, and beverages.
- Search results update according to the selected filters.

3. Cart Management Module (CMM)

- Customers can add food items to the cart and select quantities.
- Customers can update quantities or remove items from the cart.
- The system calculates the total cost based on the selected items and quantities.

4. Order Management Module (OMM)

- Customers can confirm their cart items and place food orders.
- The system stores order details in the MySQL database.
- Administrators can view orders and update their status, such as Pending, Preparing, and Delivered.

5. Customer Management Module (CuMM)

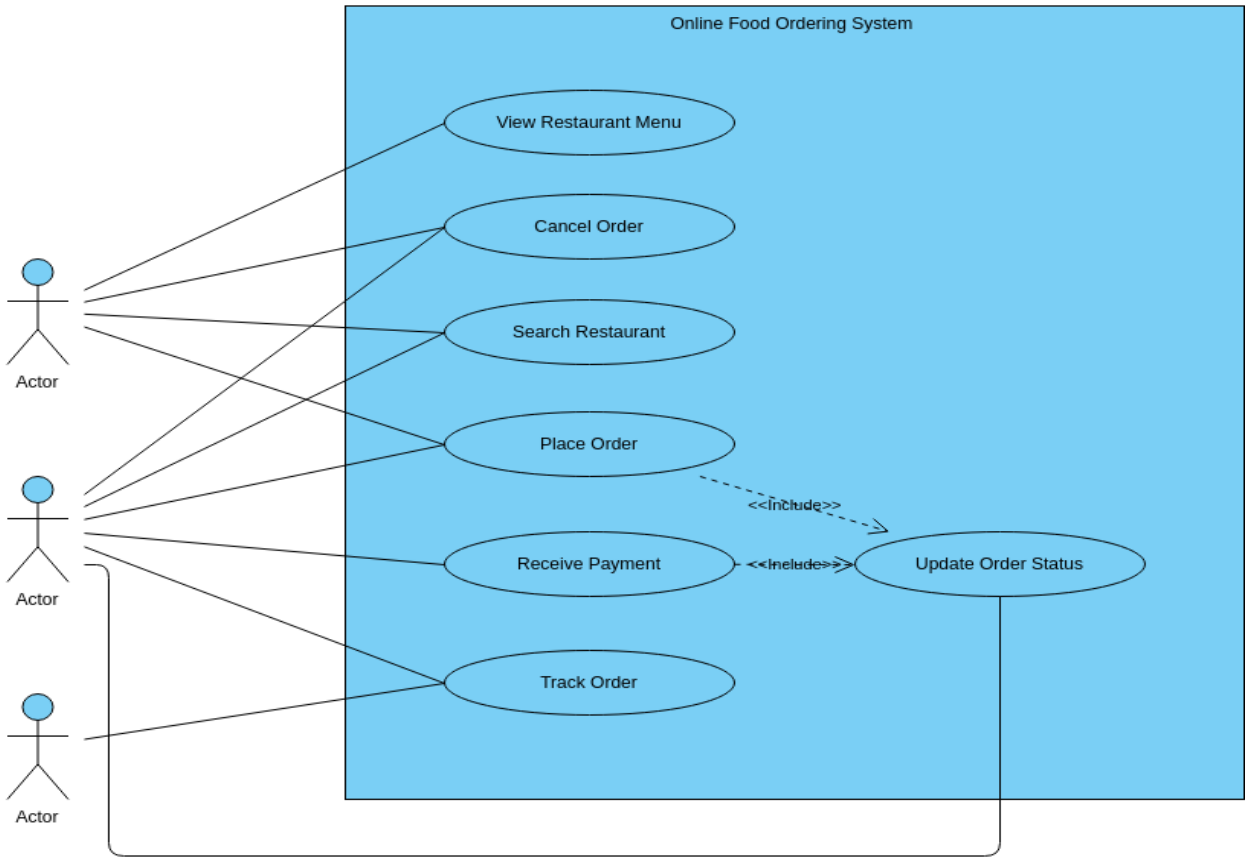
- Customers can register and log in if authentication is implemented.
- The system stores customer details and delivery addresses.
- Customers can view their order history and order information.

6. Database Management Module (DBM)

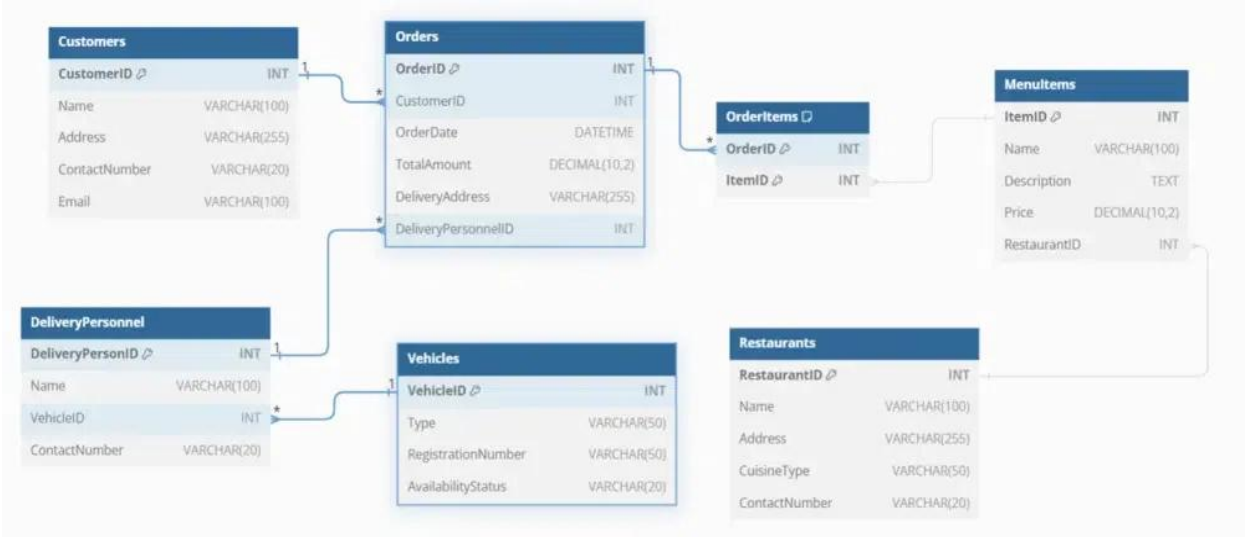
- The system connects to the MySQL database through PHP.
- The database stores food items, customer details, and order information.
- The PHP backend validates user input and processes database operations.
- The system retrieves and updates information to keep food menus and order details current.

2. SYSTEM FLOW DIAGRAMS

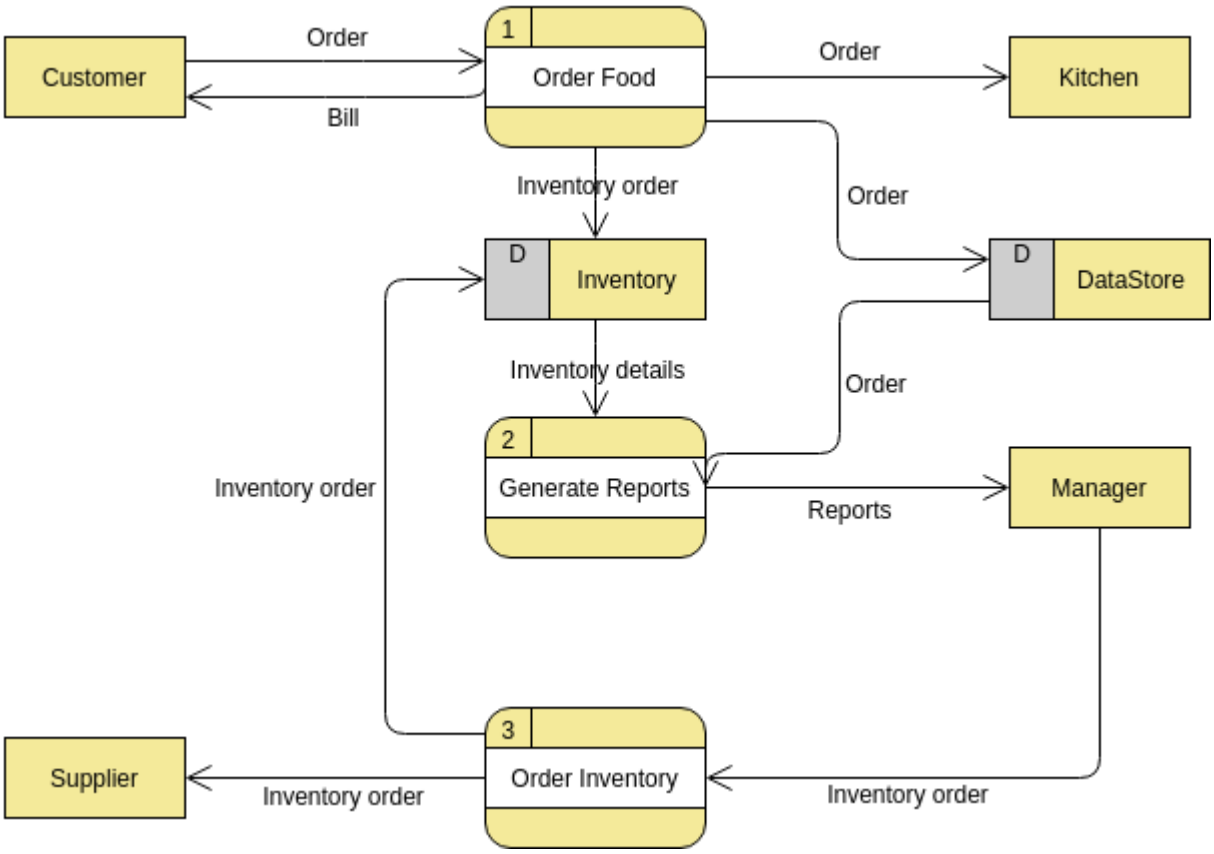
2.1 USE CASE DIAGRAM



2.2 ENTITY RELATIONSHIP DIAGRAM



2.3 DATA FLOW DIAGRAM

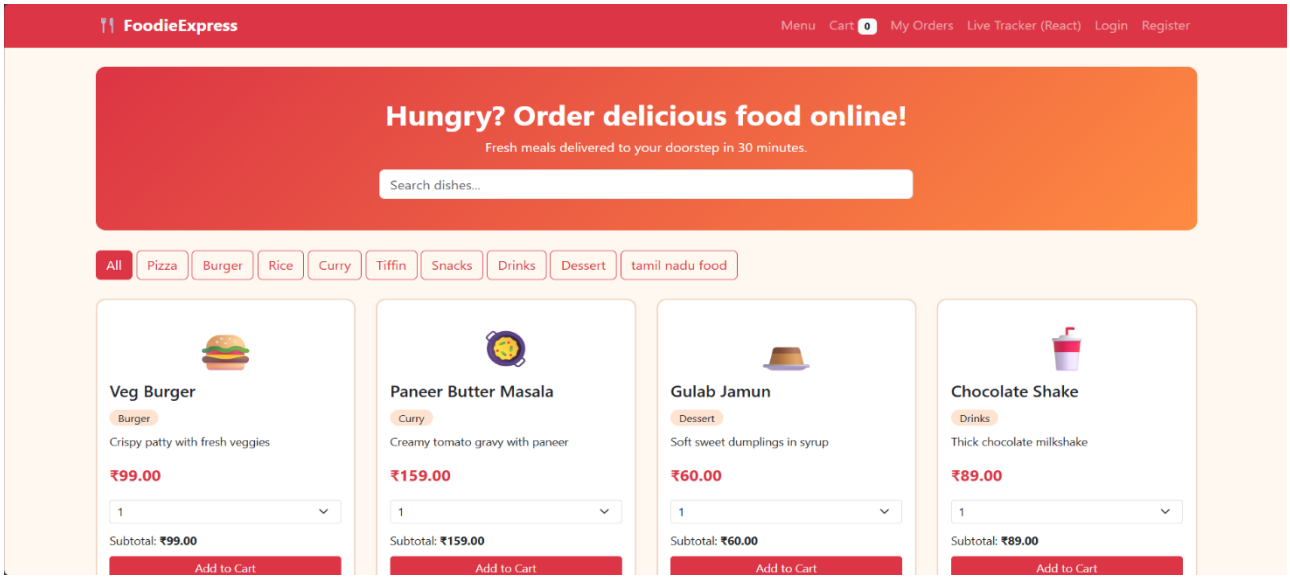


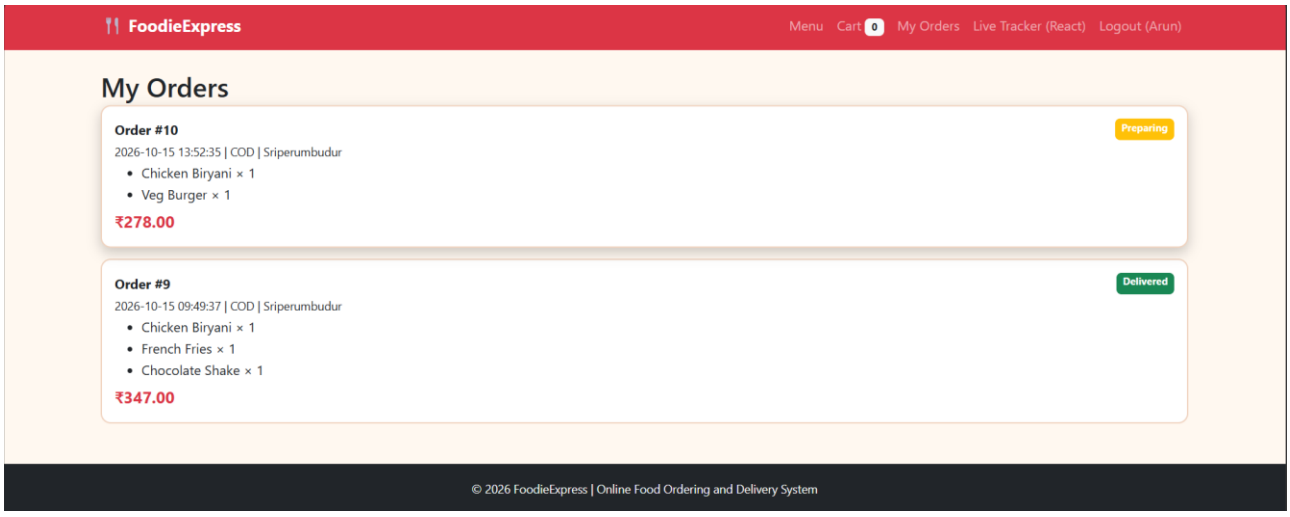
3. MODULE DESCRIPTION

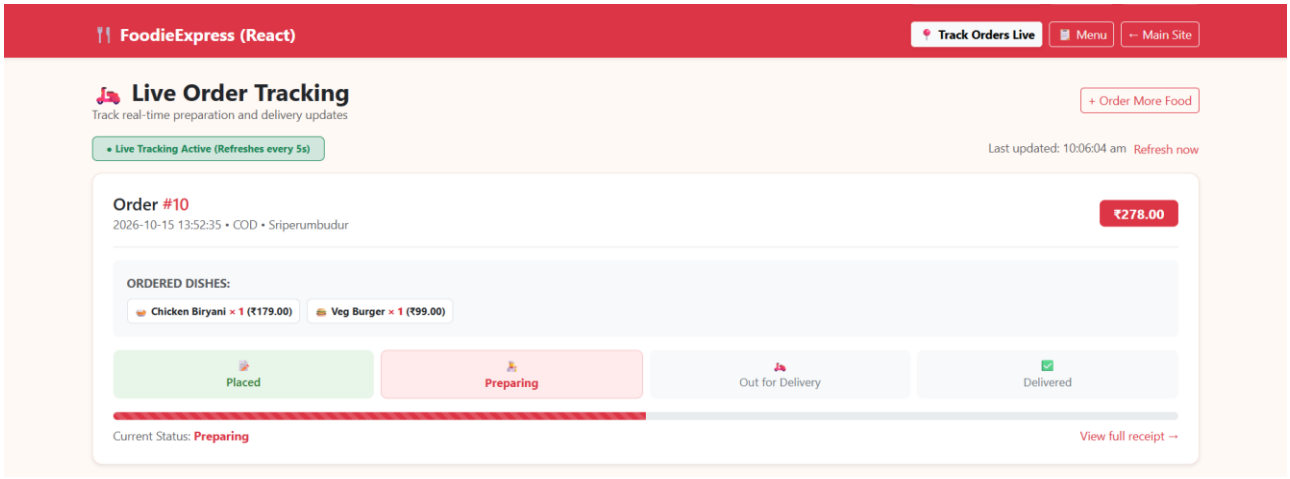
- Frontend Module (React, HTML, CSS, JavaScript): Provides the user interface for browsing food items, searching and filtering menus, viewing food details, managing the cart, placing orders, and displaying order information.
- Food Management Module: Allows administrators to add, update, delete, and view food items, including their names, descriptions, prices, categories, and images.
- Cart Management Module: Enables customers to add food items to the cart, update quantities, remove items, and calculate the total bill.
- Order Management Module: Handles customer orders, saves order details in the MySQL database, and allows administrators to view orders and update their status.
- Customer Management Module: Manages customer registration, login, profile information, and delivery addresses, if these features are implemented.

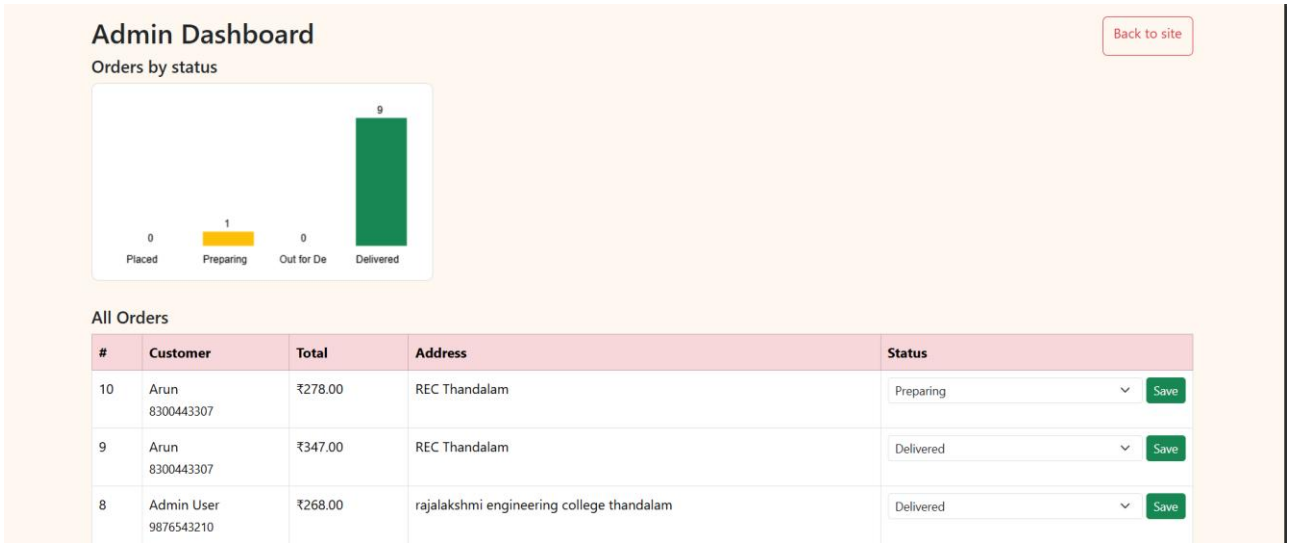
- Database Module (MySQL): Stores and retrieves customer details, food menu information, cart data, and order records. PHP manages database communication and executes the required queries.
- Backend Module (PHP): Processes requests from the React frontend, validates input, performs database operations, and returns responses to the application.
- Order History Module: Allows customers to view their previous orders and check order details and status.

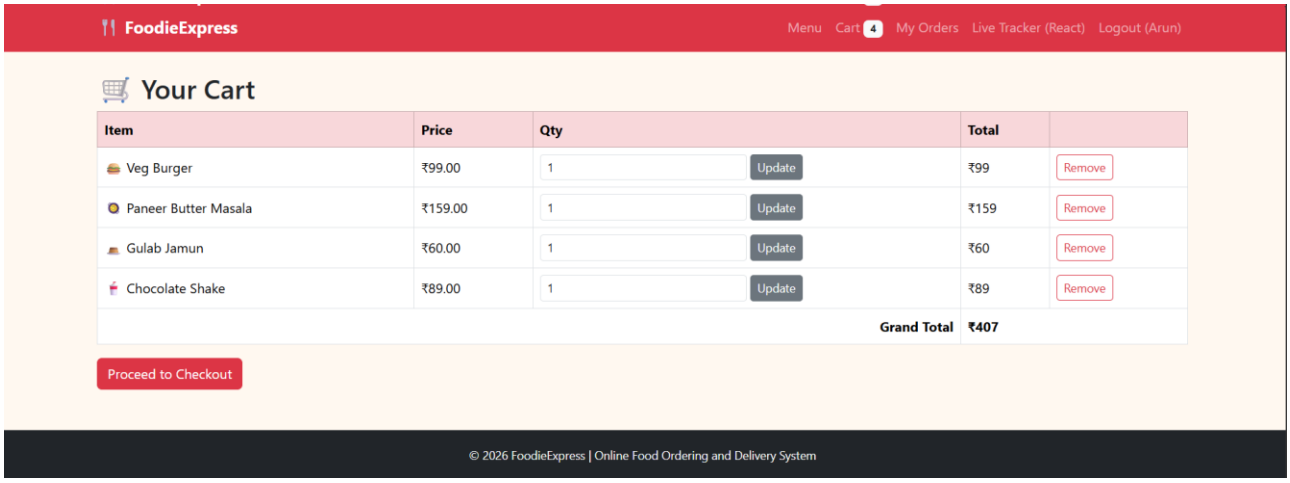
4. IMPLEMENTATION - 4.1 SCREENSHOTS






FoodieExpress (React)
Track Orders Live
Menu
← Main Site
Live Order Tracking
Track real-time preparation and delivery updates
+ Order More Food
• Live Tracking Active (Refreshes every 5s)
Last updated: 10:06:04 am Refresh now
Order #10
2026-10-15 13:52:35 • COD • Sriperumbudur
₹278.00
ORDERED DISHES:
Chicken Biryani × 1 (₹179.00)
Veg Burger × 1 (₹99.00)
Placed
Preparing
Out for Delivery
Delivered
Current Status: Preparing
View full receipt →


Admin Dashboard
Back to site
Orders by status
9
1
0
0
Placed
Preparing
Out for De
Delivered
All Orders
Customer Total Address Status
10 Arun 8300443307 ₹278.00 REC Thandalam Preparing Save
9 Arun 8300443307 ₹347.00 REC Thandalam Delivered Save
8 Admin User 9876543210 ₹268.00 rajalakshmi engineering college thandalam Delivered Save


FoodieExpress
Menu Cart 4 My Orders Live Tracker (React) Logout (Arun)
Your Cart
Item Price Qty Total
Veg Burger ₹99.00 1 Update ₹99 Remove
Paneer Butter Masala ₹159.00 1 Update ₹159 Remove
Gulab Jamun ₹60.00 1 Update ₹60 Remove
Chocolate Shake ₹89.00 1 Update ₹89 Remove
Grand Total ₹407
Proceed to Checkout
© 2026 FoodieExpress | Online Food Ordering and Delivery System

4.2 DATABASE DESIGN

The database used in the Online Food Ordering and Delivery System is MySQL. It stores customer details, food item information, and order records required for the proper functioning of the application. The database consists of tables such as customers, food_items, orders, and order_items.

The customers table stores customer information such as customer ID, name, email, password, phone number, and delivery address. The food_items table stores food item details, including food ID, food name, category, description, price, and image. The orders table maintains order information such as order ID, customer ID, order date, total amount, delivery address, and order status. including item quantity and price.

4.3 CODE

1. DATABASE CONNECTION AND TABLE CREATION (api/db.php)

```
<?php

$DB_HOST = '127.0.0.1';
$DB_PORT = '3306';
$DB_USER = 'root';
$DB_PASS = '********';
$DB_NAME = 'food_ordering';

try {
    $conn = new PDO(
        "mysql:host=$DB_HOST;port=$DB_PORT;dbname=$DB_NAME;charset=utf8mb4",
        $DB_USER,
        $DB_PASS
    );

    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $conn->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
    $conn->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);

    $conn->exec("
        CREATE TABLE IF NOT EXISTS customers (
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(100) NOT NULL,
            email VARCHAR(150) NOT NULL UNIQUE,
            password VARCHAR(255) NOT NULL,
            phone VARCHAR(20),
            address TEXT,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    ");

    $conn->exec("
        CREATE TABLE IF NOT EXISTS food_items (
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(100) NOT NULL,
            description TEXT,
```

```
            price DECIMAL(10,2) NOT NULL,
            category VARCHAR(100),
            image VARCHAR(255),
            availability TINYINT(1) DEFAULT 1,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    ");

    $conn->exec("
        CREATE TABLE IF NOT EXISTS orders (
            id INT AUTO_INCREMENT PRIMARY KEY,
            customer_id INT NULL,
            customer_name VARCHAR(100) NOT NULL,
            address TEXT NOT NULL,
            phone VARCHAR(20),
            total DECIMAL(10,2) NOT NULL,
            status ENUM(
                'Pending',
                'Confirmed',
                'Preparing',
                'Out for Delivery',
                'Delivered',
                'Cancelled'
            ) DEFAULT 'Pending',
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (customer_id)
                REFERENCES customers(id)
                ON DELETE SET NULL
        )
    ");

    $conn->exec("
        CREATE TABLE IF NOT EXISTS order_items (
            id INT AUTO_INCREMENT PRIMARY KEY,
            order_id INT NOT NULL,
            food_id INT NOT NULL,
            food_name VARCHAR(100) NOT NULL,
            quantity INT NOT NULL,
            price DECIMAL(10,2) NOT NULL,
            subtotal DECIMAL(10,2) NOT NULL,
            FOREIGN KEY (order_id)
                REFERENCES orders(id)
                ON DELETE CASCADE,
            FOREIGN KEY (food_id)
                REFERENCES food_items(id)
        )
    ");

} catch (PDOException $e) {
    die("Database connection failed.");
}
?>
```

2. FETCH, SEARCH AND FILTER FOOD ITEMS (api/food.php)

```
<?php

require 'db.php';

header('Content-Type: application/json');

$method = $_SERVER['REQUEST_METHOD'];

if ($method === 'GET') {
```

```
    $search = trim($_GET['search'] ?? '');
    $category = trim($_GET['category'] ?? 'All');

    $sql = "SELECT * FROM food_items WHERE availability = 1";
    $params = [];

    if ($search !== '') {
        $sql .= " AND (name LIKE :search OR description LIKE :search)";
        $params[':search'] = '%' . $search . '%';
    }

    if ($category !== '' && $category !== 'All') {
        $sql .= " AND category = :category";
        $params[':category'] = $category;
    }

    $sql .= " ORDER BY id DESC";

    try {
        $stmt = $conn->prepare($sql);
        $stmt->execute($params);

        echo json_encode([
            "success" => true,
            "count" => $stmt->rowCount(),
            "food_items" => $stmt->fetchAll()
        ]);

    } catch (PDOException $e) {
        http_response_code(500);

        echo json_encode([
            "success" => false,
            "message" => "Unable to fetch food items."
        ]);
    }

    exit;
}

if ($method === 'POST') {

    $data = json_decode(file_get_contents("php://input"), true);

    $name = trim($data['name'] ?? '');
    $description = trim($data['description'] ?? '');
    $price = $data['price'] ?? 0;
    $category = trim($data['category'] ?? '');
    $image = trim($data['image'] ?? '');

    if ($name === '' || !is_numeric($price) || $price <= 0) {
        http_response_code(400);

        echo json_encode([
            "success" => false,
            "message" => "Enter a valid food name and price."
        ]);

        exit;
    }

    $stmt = $conn->prepare("
        INSERT INTO food_items
        (name, description, price, category, image)
```

```
        VALUES (?, ?, ?, ?, ?)
    ");

    $stmt->execute([
        $name,
        $description,
        $price,
        $category,
        $image
    ]);

    echo json_encode([
        "success" => true,
        "message" => "Food item added successfully.",
        "food_id" => $conn->lastInsertId()
    ]);

    exit;
}
```

3. UPDATE AND DELETE FOOD ITEMS (api/manage_food.php)

```
<?php

require 'db.php';

header('Content-Type: application/json');

$method = $_SERVER['REQUEST_METHOD'];

if ($method === 'PUT') {

    $data = json_decode(file_get_contents("php://input"), true);

    $id = (int)($data['id'] ?? 0);
    $name = trim($data['name'] ?? '');
    $description = trim($data['description'] ?? '');
    $price = $data['price'] ?? 0;
    $category = trim($data['category'] ?? '');
    $image = trim($data['image'] ?? '');

    if ($id <= 0 || $name === '' ||
        !is_numeric($price) || $price <= 0) {

        http_response_code(400);

        echo json_encode([
            "success" => false,
            "message" => "Invalid food item details."
        ]);

        exit;
    }

    $stmt = $conn->prepare("
        UPDATE food_items
        SET name = ?,
            description = ?,
            price = ?,
            category = ?,
            image = ?
        WHERE id = ?
    ");
```

```
    $stmt->execute([
        $name,
        $description,
        $price,
        $category,
        $image,
        $id
    ]);

    echo json_encode([
        "success" => true,
        "message" => "Food item updated successfully."
    ]);

    exit;
}
```

4. LOAD AND DISPLAY FOOD ITEMS IN REACT (App.jsx)

```
import React, { useEffect, useState } from 'react';

function App() {
    const [foodItems, setFoodItems] = useState([]);
    const [search, setSearch] = useState('');
    const [category, setCategory] = useState('All');
    const [cart, setCart] = useState([]);
    const [loading, setLoading] = useState(false);
    const [message, setMessage] = useState('');

    const API = 'http://localhost/food-ordering/api';

    const fetchFoodItems = async () => {
        try {
            setLoading(true);

            const params = new URLSearchParams({
                search: search,
                category: category
            });

            const response = await fetch(
                `${API}/food.php?${params.toString()}`
            );

            const data = await response.json();

            if (data.success) {
                setFoodItems(data.food_items);
            }
        } catch (error) {
            setMessage('Unable to load food items.');
        } finally {
            setLoading(false);
        }
    };

    useEffect(() => {
        fetchFoodItems();
    }, [search, category]);

    const addToCart = (food) => {
        setCart(previousCart => {
            const existing = previousCart.find(
                item => item.id === food.id
            );

            if (existing) {
                return previousCart.map(item =>
```

```
                    item.id === food.id
                        ? { ...item, quantity: item.quantity + 1 }
                        : item
                );
            }

            return [
                ...previousCart,
                { ...food, quantity: 1 }
            ];
        });

        setMessage('Food item added to cart.');
    };

    const updateQuantity = (id, quantity) => {
        if (quantity < 1) {
            setCart(cart.filter(item => item.id !== id));
            return;
        }

        setCart(cart.map(item =>
            item.id === id
                ? { ...item, quantity: quantity }
                : item
        ));
    };

    const removeFromCart = (id) => {
        setCart(cart.filter(item => item.id !== id));
    };

    const total = cart.reduce(
        (sum, item) => sum + Number(item.price) * item.quantity,
        0
    );
```

5. PLACE ORDER AND SAVE ORDER DETAILS (api/order.php)

```
<?php

require 'db.php';

header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);

    echo json_encode([
        "success" => false,
        "message" => "Method not allowed."
    ]);

    exit;
}

$data = json_decode(file_get_contents("php://input"), true);

$name = trim($data['customer_name'] ?? '');
$address = trim($data['address'] ?? '');
$phone = trim($data['phone'] ?? '');
$items = $data['items'] ?? [];

if ($name === '' || $address === '' || empty($items)) {
    http_response_code(400);

    echo json_encode([
        "success" => false,
        "message" => "Enter customer details and select food items."
```

```
        ]);

        exit;
    }

    try {
        $conn->beginTransaction();

        $total = 0;
        $orderItems = [];

        $foodStmt = $conn->prepare("
            SELECT id, name, price
            FROM food_items
            WHERE id = ? AND availability = 1
        ");

        foreach ($items as $item) {

            $foodId = (int)($item['id'] ?? 0);
            $quantity = (int)($item['quantity'] ?? 0);

            if ($foodId <= 0 || $quantity <= 0) {
                throw new Exception("Invalid food item or quantity.");
            }

            $foodStmt->execute([$foodId]);
            $food = $foodStmt->fetch();

            if (!$food) {
                throw new Exception("Food item is unavailable.");
            }

            $price = (float)$food['price'];
            $subtotal = $price * $quantity;

            $total += $subtotal;

            $orderItems[] = [
                "id" => $food['id'],
                "name" => $food['name'],
                "price" => $price,
                "quantity" => $quantity,
                "subtotal" => $subtotal
            ];
        }

        $orderStmt = $conn->prepare("
            INSERT INTO orders
            (customer_name, address, phone, total, status)
            VALUES (?, ?, ?, ?, 'Pending')
        ");

        $orderStmt->execute([
            $name,
            $address,
            $phone,
            $total
        ]);

        $orderId = $conn->lastInsertId();

        $itemStmt = $conn->prepare("
            INSERT INTO order_items
            (order_id, food_id, food_name, quantity, price, subtotal)
            VALUES (?, ?, ?, ?, ?, ?)
        ");
```

5. CONCLUSION

The Online Food Ordering and Delivery System replaces traditional food ordering methods with a user-friendly, database-backed web application. It combines a React interface, a PHP backend and a MySQL database to allow customers to browse food items, add items to the cart, place orders and view order details. The project provided practical experience in component-based UI design, backend development, database management and full-stack integration. Future improvements include online payment integration, real-time order tracking, email or SMS notifications, customer login, delivery partner management and deployment on a live server.

6. REFERENCES

1. React Documentation, react.dev
2. PHP Manual (PDO and built-in web server), php.net
3. MySQL 8.0 Reference Manual, dev.mysql.com
4. MDN Web Docs, *Fetch API*, developer.mozilla.org
5. Babel Standalone Documentation, babeljs.io
6. Visual Studio Code Documentation, code.visualstudio.com